

כתיבת Test Cases, תרחישים ועקיבות

A – אנטומיית מקרה בדיקה

- **Preconditions | תנאים מקדימים:** תנאים מקיימים שעל המערכת או בסביבה לקבל מענה לפני התחלת הבדיקה. למשל, משתמש רשום במערכת או נתוני בדיקה קיימים במסד. תפקידם לוודא שמבחן ניתן להרצה (לדוגמה, יצירת משתמש/הרשמה מראש) – כדי שמקרה הבדיקה יהיה עצמאי. **K: K2** (הבנה). **זווית מבחן:** נבחן למשל כיצד מבדילים בין התייחסות ל־precondition לבין שלב הצעד הראשון בבדיקה, או האם לא בשלב ההרצה. **טעות שכיחה:** השמטת תנאים חשובים (למשל מניחים שהמשתמש כבר מחובר) או גיבוב תנאים מכליל בגוף מקרה הבדיקה במקום לשים אותם כהקדמה.
- **Steps (Test Steps) | צעדים / שלבי בדיקה:** סדרת פעולות מפורטת שעל הבודק לבצע במהלך הבדיקה. על כל שלב להיות ממוספר ולבצע פעולה אחת ברורה (למשל: הזן שם משתמש). **K: K2**. **זווית מבחן:** נבדק למשל דיוק הפירוט והאם כל שלב נשאל בסדר הנכון. **טעות שכיחה:** כתיבה לא ברורה או קצרה מדי, למשל שילוב פעולות מרובות באותו שלב ("נווט לדף ההגדרות ועדכן פרופיל"), או מתן מידע חסר כמו "התחבר" בלי לציין שם משתמש.
- **Test Data | נתוני בדיקה:** ערכי קלט מוגדרים ומשתנים שיש להשתמש בהם בהרצת המקרה. לדוגמה, פרטי חשבון, מספרים קיצוניים, ערכים תקפים ותוקפים. המטרה היא למנוע דו-משמעות בביצוע ולהבטיח עקביות (דהיינו, נתונים ספציפיים מוכנים מראש) **[L7-L10+56]**. **K: K2**. **זווית מבחן:** מתמקדים בפיזור הנתונים ולא בשיבוץ ישיר בצעדים. בבחינה ניתנת דוגמה של מקרה בדיקה עם נתונים מצורפים או תוך כדי צעד, ונשאל אם זה טוב (הפרדה טובה מאפשרת שימוש חוזר בנתונים במספר מקרים). **טעות שכיחה:** הגדרת נתונים בתוך הטקסט של הצעד במקום לשמור אותם כפראמטר (למשל "הזן שם משתמש וסיסמה" ללא ציון הערכים עצמם), או היעדר רשימת נתונים חלופית (כגון מבחר ערכים ל-edge cases). תלות בנתונים חיצוניים (כמו נתונים שמושחזים ב־DB) עלולה לגרום לשבריריות: שינוי בנתונים יגרום לכישלון המבחן.
- **Expected Result | תוצאה צפויה:** התוצאה המדויקת שיש לאמת לכל צעד או בסיום הבדיקה. תיאור תוצאה זו במילים ברורות (למשל: "האתר יציג הודעת שלום עם שם המשתמש" או "מופיעה הודעת שגיאה המתארת בעיית סיסמה"). **K: K2**. **זווית מבחן:** נבדק שה־expected לא חוזר על שלב הפעולה (אין לשכתב את השלב כהסבר), ושהוא ממוקד ונבדק על ידי בדיקה אוטומטית או ידנית. **טעות שכיחה:** ניסוח כללי או עקום של התוצאה ("האתר נטען" במקום "נטען דף הבית עם נתוני המשתמש הנכונים"), או שכפול של הצעד עצמו ("נלחץ לחצן הכניסה" במקום מה אמור לקרות בעקבותיו) **[L85-L93+59]**.
- **Postconditions | תנאים לאחר הרצה:** מצב המערכת המצופה בסיום מקרה הבדיקה (למשל, משתמש מחובר או נתונים נרשמו בבסיס). משמש להבטיח שכל שינוי יוחזר למצב התחלתי להתקפה של מבחן הבא או להמשך שרשרת מבחנים. **K: K2**. **זווית מבחן:** בבחינה נבחן אם מציינים בבירור מה מצופה לאחר ריצה (לדוגמה: האם נדרשת יציאה מהמערכת לאחר סוף מבחן?). **טעות שכיחה:** הזנחת ציון של תנאי סיום (למשל נשארים מחוברים בטעות) או תלות בלתי-צודקת בכך שהבודק "יודע" שיש לנקות מצב (קבצים זמניים, נתונים טמאים) ללא שתתועד זו בחלק האחרון.

B – מבחן מול תרחיש ולחילוקי מושגים

- **Test Condition | תנאי בדיקה:** פריט או אירוע במערכת הניתן לאימות על-ידי מקרה/מקרים של בדיקה – למשל פונקציה, טרנזקציה או תכונה ספציפית **[L3800-L3804+36]**. זהו למעשה אפיון גבוה של מה צריך להיבדק (למשל "רישום משתמש חדש"). **K: K2**. **זווית מבחן:** מושווה למשל לדרישת מערכת ולאופן כתיבת

המקרים: האם הגדרה של תנאי בדיקה נכונה כבודקת פונקציה אחת ברורה. **טעות שכיחה:** בלבול עם "מקרה בדיקה" עצמו, או הגדרת תנאי כללי מדי (למשל "המערכת עובדת" במקום פעולה ברורה) [L3800-L3804+36].

• **Test Case | מקרה בדיקה:** תיאור מפורט של בדיקה אחת, הכולל תנאי התחלה, סדרת צעדים ברורים (עם הנתונים) ותוצאה צפויה בודקת. זהו היחידה הקטנה ביותר של בדיקה תיעודית. דוגמה: מקרה בדיקה לוודא כניסה תקינה עם שם משתמש וסיסמה תקינים. **K: K2-K3. זווית מבחן:** נבדל ממבחן אוטומטי (test script) ומתרחיש בכך שמסביר בדיוק מה לעשות; בשאלה יבחנו למשל האם מופיעים בכל מקרה כל שדות מבנה נכונים (להבדיל מלימוד על הגדרה נכונה של מקרה בדיקה). **טעות שכיחה:** ניסוח לא עקבי (מילים כלליות מדי), חוסר בנתונים במפורש, ריבוי של פעולות בשלב אחד. בנוסף, בלבול עם **תרחיש** או **תסריט:** למשל לגבי מה ברמת הפרטנות (מקרה בדיקה הוא רמת ביצוע מפורטת).

• **Test Scenario | תרחיש בדיקה:** תיאור ברמת על של זרימת בדיקה או תרחיש עסקי רחב, המאגד מספר מקרים עוקבים. לרוב זהו "סיפור" היפותטי מתמשך שאינו מפורט מאוד, ומכסה כמה שלבים עיקריים [L7+48-L10]. תרחיש משמש לבדיקות מורכבות או אינטגרציה (למשל "רישום משתמש וביצוע רכישה רצופה"). הם שונים ממקרה בדיקה בכך שהתמקדותם כוללת מספר צעדים ועיקוב של תרחיש שלם, ולא מבצעים פריט יחיד. **K: K3. זווית מבחן:** בבחינה עשויים לבדוק זיהוי תרחיש מתאים בתוכנית בדיקות מול מקרים. **טעות שכיחה:** כתיבה של תרחיש ברמת פרטנות נמוכה מדי או להפך לבלבל אותו עם מקרה בדיקה בודד. למשל זיהוי ש"תרחיש" עם רשימת צעדים מפורטת (לפעמים יוצרת בלבול עם **test case**).

• **Test Script | תסריט בדיקה:** קובץ או מסמך המיועד בעיקר לבדיקות אוטומטיות, המתאר סדרת צעדים או מקרים לביצוע (כולל פעולות הכנה ותסריטים לפתיחה/סיום). במובן הרחב, המונח נקשר לביצוע סקריפט לבדיקת מוצר (ולעיתים נרדף עם "פרוצדורת בדיקה אוטומטית"). ברמת ידנית הוא דומה ל"טבלת בדיקה" (test procedure) שבה מופיעים מספר מקרים לפי סדר הרצה [L4171-L4180+40]. **K: K2. זווית מבחן:** במבחן אפשר לשאול מה ההבדל מ-mחקרא בדיקה: למשל "תסריט בדיקה נועד לבדיקה אוטומטית, מקרה בדיקה הוא לרוב ידני ומפורט". **טעות שכיחה:** להשתמש ב"תסריט" כשמדובר ב-"מקרה בדיקה" פשוט, או לפעמים להפך; או חוסר ציון שעורך הבדיקה צריך לדעת שזה אוטומטי.

• **char | Test Charter בדיקה:** הנחיית בדיקה קצרה (מטרת בדיקה) המכוונת ששן בדיקה חקרנית (Exploratory). זוהי הצהרת מטרה ורעיון כללי איך לבדוק משהו ספציפי [L3769-L3772+42]. למשל: "בדוק את טופס הכניסה כדי לגלות חולשות אבטחה". **K: K3. זווית מבחן:** בבחינה שואלים מה צריך לכלול צ'ארטר (מטרות ותחום סיכון) וכיצד הוא שונה ממקרה בדיקה מפורט. **טעות שכיחה:** ניסוח כללי מדי ("בדוק פונקציונליות הרשמה" בלי פירוט סיכון) או להפך לכתוב את תכנית הבדיקה כולה במקום מטרה. רבים מבלבלים אותו עם "מבחן לחיץ אוטומטי" או "תוכנית בדיקה" – אך הבדל המרכזי שבצ'ארטר לא מפרטים צעדים אלא רק מטרות וחזון כללי [L3769-L3772+42] [L45+L6-L14].

• **High-Level vs Low-Level Test Case:** **מקרה בדיקה גבוה** (High-Level) הוא טיוטת מקרה כללית ללא ערכים ספציפיים (למשל "בדוק כניסה תקינה" עם משמעות כללית). **מקרה בדיקה נמוך** (Low-Level) מפרט את כל הערכים והצעדים בדיוק (למשל "הזן יוסי, סיסמה 123" ולחץ כניסה). High-level מתאים לבדיקת תכנון (וגם אוטומציה עולה ממנו לפרמטרים), בעוד Low-level הוא מוכן לביצוע ישיר. במבחן עשויים לשאול מתי לבחור כל אחד: High-Level כשצריך סקירה מהירה או חיפוש בעיות כלליות; Low-Level כשמעוניינים בפרודוסביליות מדויקת [L3800-L3804+36] [L40+L4171-L4180]. **טעות נפוצה** היא לערבב ביניהם (למשל לכתוב שלב אחד כפלטפורמה לבדיקות נרחבות) או לכתוב High-Level כמו Low-Level ולהפך.

C – נתוני בדיקה

• **Test Data Design | עיצוב נתוני בדיקה:** תהליך שבו מגדירים מראש את מערכי הקלט לכל מקרה בדיקה, באופן נפרד מהצעדים עצמם. הנתונים צריכים להיות רלוונטיים ומגוונים (תקינים, שגויים, גבוליים), כדי לכסות את תחומי הקלט של הפונקציה [L7-L10+56]. שימוש חוזר בנתונים (למשל מאגר מטה-נתונים מרכזי) חוסך עבודה ומבטיח עקביות בין מקרים. **K: K3. זווית מבחן:** בבחינה ישאלו מדוע מפרידים בין "שלב הבדיקה" ו"נתוני הקלט" (לתכנן כמו מסד נתונים של בדיקות), ואיך תלות בנתונים חיצוניים גורמת לשבריריות הבדיקה. **טעות**

שכיחה: לא להגדיר ממדיאלית את התלות בנתוני הקלט (למשל: "נחבר למערכת לקוח ספציפי" בלי לציין מה המזהה שלו), או לשבץ נתונים "אקראיים" בתוך הצעד ("שמור שם לקוח אוטומטי" ללא הערך עצמו). גם אי שימוש באותם נתונים למבחנים דומים נחשב לבזבז. Dependency בנתונים (למשל הנחה שיש מנוי בספציפי) הוא מקור לשבירה: שינוי נתונים בשרת יעצור את המבחן.

D - תרחישים ותלויות בין מקרים

• **Scenario Chaining | שרשור תרחישים (Test Scenarios):** כשמספר מקרים מצורפים זה לזה בסדר מסוים, כך שהתוצאה של אחד היא תנאי ההתחלה של הבא (לדוגמה: "בדוק רישום משתמש חדש" → "בדוק כניסה עם המשתמש הזה"). במילים אחרות, תרחיש בדיקה מורכב מכמה מקרים עוקבים המייצרים מצב רציף במערכת. **K:K3. זווית מבחן:** נבחן מתי כדאי להשתמש בשרשור (למשל בבדיקות end-to-end ורציפות) ומתי לא (למשל בבדיקות איטרטיביות עצמאיות). **טעות שכיחה:** יצירת תלות מיותרת: כתיבת מבחנים שלא ניתן להריץ בנפרד (למשל אם מקרה אחד נכשל - כל השרשרת נכשלה) במקום לשמור על עצמאות מקרים.

• **Dependencies | תלות בין מקרים:** מצב שבו מקרה בדיקה אחד מסתמך על מצב שהושג על-ידי מקרה אחר. תלות זו יכולה להיות לגיטימית (כאשר יש קשר לוגי ברור, כמו התחברות לפני פעולה מסוימת) אך לעיתים היא מלכודת: אם היא כתובה סמונה, המבחנים עשויים להיות מושפעים זה מזה. **K:K3. זווית מבחן:** בבחינה עשויים להראות רצף מקרים ולשאול איזו תלות קיימת והאם ניתן לארגן אותם אחרת (למשל על-ידי עיצוב טוב יותר של preconditions). **טעות שכיחה:** תלות נסתרת או לא מתועדת ("מקרה הבדיקה מניח שמשתמש כבר מחובר למערכת"), מה שמוביל ל"תלות בשכל" של מחבר התסריט. זאת עשויה להיות מלכודת - מומלץ לתעד תלות מפורשות (למשל בסדר הריצה) או לפרק את השרשור.

E - עקיבות (Traceability)

• **Traceability Matrix | מטריצת עקיבות דרישות-בדיקות:** טבלה דו-ממדית המקשרת בין דרישות (או user stories) למקרי בדיקה שבודקים אותן. מטריצה זו מאפשרת לעקוב שבסיסה של כל מקרה בדיקה יש דרישה רלוונטית, ולהבטיח שכל דרישה מכוסה על-ידי לפחות מקרה בדיקה **[L315-L323+18]**. שמירת עקיבות היא הכרחית למשל בתעשיות מבוקרות, למניעת תקלות ולהוכחת כיסוי בדיקות (ניתן לראות אילו דרישות חסרות כיסוי, וכן מבחנים שהייתה להם דרישה).

• **Coverage Gaps (דרישות בלי כיסוי / בדיקות בלי דרישות):** בבדיקות עקיבות מתייחסים לשני כיוונים: *Forward traceability* (כל דרישה מחוברת לבדיקה) ו-*Backward traceability* (כל מקרה בדיקה מקושר לדרישה). בחינת הפערים מצביעה על "דרישות לא נבדקו" או "בדיקות מיותרות". חסר תקינות עקיבות - תסריטים שאינם קשורים לדרישה - מסמנים בדיקות ארכאיות או לא רלוונטיות. **K:K2-K3. זווית מבחן:** נשאל האם ישנה דרישה ללא מקרה (חסר כיסוי) או מקרה שאין לו דרישה (בדיקה מוטלת סתם). למשל "נמצא דרישה לשמירת סיסמה אך אין לה מקרה בדיקה תואם".

• **Impact Analysis (ניתוח השפעה):** עקיבות משמשת לניתוח שינויים: כאשר דרישה משתנה, אנו משתמשים במטריצת העקיבות כדי לזהות במהירות אילו מבחנים מושפעים מהשינוי. כך חוסכים זמן ובאגים מתישים מאוחר יותר **[L351-L354+18]**. **טעות שכיחה:** אי שימוש במטריצה - אם צוות לא מנהל עקיבות, שינוי נדרש עלול להחמיץ מבחנים חשובים, או לבצע מבחנים שאינם נדרשים כלל.

F - תחזוקת מאגר מקרי הבדיקה

• **Rework after Changes (עדכון אחרי שינוי):** כל שינוי בדרישה או במערכת מצריך בדיקה ועדכון של מקרי הבדיקה המתאימים. מיומנות הבדוק היא לזהות אילו מקרים יש לתקן (למשל עדכון expected או שינוי בנתוני הקלט) ומתי ליצור חדשים. **K4 (יישום). טעות שכיחה:** לא לעדכן מקרים (למשל כשהוסיפו שדה חדש, ועדיין כותבים תוצאה צפויה ישנה) - זה יוצר "ריקבון" בתיק הבדיקות.

• **Signs of Decay (סימני ריקבון):** ריכוז מקרים גדל מעבר לצורך, והם מחמירים את חוסר היעילות. סימנים: מקרים כפולים (ניסוחים זהים), תוצאות צפויות מעורפלות, מקרים שלא בוצעו אף פעם, או כאלה שתלויות במידע בראש הכותב. **K: K3.** זווית מבחן: בבחינה ייתכן תרחיש של ריכוז בטסט לרשימות ארוכות ושאלה אילו לקצר, לאחד או למחוק (למשל "בנקודות 3 ו-7 זהה – איחד/מחק" או "תוצאה צפויה כללית מדי – פרט אותה"). טעות נפוצה נוספת היא **חוסר תחזוקה של exit criteria**: לאסוף קריטריוני יציאה אובייקטיביים (כמות בלוגים נסרקים, כמות בדיקות שעברו) ולכן הם נוצרים באופן מילולי ומשתמע.

תוצרים רוחביים ותרגילים

• **טעויות שטח (12-14 טעות):** לדוגמה: תוצאה צפויה החוזרת על הצעד הנבדק (טעות בניסוח); צעדים ללא ערכי קלט (מופיע "הזן שם משתמש" ללא ציון המשתמש או ערך הסיסמה); המקרה תלוי בידע שלא צוין ("המשתמש כבר מחובר"); ניסוח צ'ארטר במילים כלליות ("בדוק רכישת מוצר" ללא פירוט סיכון); ציון Exit Criteria תכולת מילים (מקשים בדו"חות הצלחות/כישלונות – במקום מדדים כמותיים) [L85-L93+59] [L106-L113+59].

• תרגילים אופייניים (5-7 מבנים):

• **דרישה קצרה → כתבי 3 מקרים מלאים** (למשל, הדרישה: "לאפשר התחברות עם מייל וסיסמה תקפים; במקרה של שגיאה תופיע הודעה רלבנטית."). על הפורמט לכלול תנאי התחלה (למשל יצירת משתמש מראש), צעדים, נתונים ותוצאה צפויה.

• **תיקון מקרה גרוע:** נותנים מקרה בדיקה שנכתב רע (למשל ללא ערכים, צעד אחד משלב פעמיים, או תוצאה כללית) ובקשה לתקן את הליקויים (לזהות מה חסר או שגוי ולנסח מחדש בשלמותו).

• **שרשור מקרים/תרחיש:** מציגים רצף מקרים ובוחנים האם הביצוע בסדר נכון (לדוגמה, אם משתמש צריך להתחבר לפני שליחת טופס). שואלים לזהות אם יש תלות ומשהו לסדר או לשנות.

• **מילוי מטריצת עקיבות:** נותנים רשימת דרישות ומקרי בדיקה חלקיים; התלמיד צריך להתאים ביניהם (למטה-מטה) וגם להצביע על דרישה ללא מקרה או מקרה מיותר.

• **יצירת תסריט לחווית משתמש:** למשל, "נתון תרחיש משתמש בראש ובראשונה – הסבר מה יבדוק במבחן (תסריט אוטומטי או בידני) ומה יהיו התנאים והצעדים." (מתאים למילוי תפקיד או טאסק עשיר).

• **המרת דרישה לסטוריז בדיקה:** קובעים כמה תרחיש על ודאי, ותוך כדי זיהוי של תנאים, הנתונים והצפי לכל אחד.

• **כתיבת Exit Criteria:** מציגים תכנית בדיקה עם אוסף טפסים, ושואלים לנסח קריטריון יציאה מדיד (למשל "90% מקרי הבדיקה עברו, או לא נותרו באגים קריטיים").

• **סגנון שאלות מבחן:** בדרך כלל שימוש במצבי קיצון (מלכודות) שבהם סימון אחד לא נכון מושך תשומת לב. למשל: "איזה מהבאים אינו חלק ממקרה בדיקה תקין?" (עם תשובות – בדוק שהתשובה היא Test Scenario ולא Test Data למשל). "מה נכון לגבי תרחיש בדיקה לעומת מקרה בדיקה?" – מיון או השלמת הצהרה תוך הימנעות מהגדרות מעורפלות. בפועל בוחנים זיהוי החלק הנכון של מבנה (כמו שימוש בצ'ארטר במבחן חקרני) והבדלים ביניהם.

• **הקשר ישראלי:** בראיונות QA נפוץ לקבל משימה מעשית: "כתוב מקרה בדיקה לדרישה קצרה זו" או "בדוק את מקרה הבדיקה הבא ותקן אותו". לדוגמה, בשאלות של John Bryce נאמר: "איך היית כותב מסמך בדיקות (STD) או מקרי בדיקה (Test Cases) לפיצ'ר חדש?" [L1-L4+60]. כמו כן מתבקשים סטודנטים או בדיקות להציג תנאים ושידות בעברית ובאנגלית – חשוב להכיר מונחים רלוונטיים (Preconditions = תנאים מקדימים, Expected result = תוצאה צפויה, Test Script = תסריט בדיקה). בעבודות ביקורת אמורות לסמן שימוש בדאטה ראיסטית ובדוחות מילוליים תקינים.

• **מקורות ורישוי:** החומר נעזר במקורות מקצועיים, כולל ספרות MoT-BBST (CC-BY) ואנציקלופדיות מבחן. רשימת מקורות (כוללת URL, מוציא לאור, שנה, ושורת רישיון בדיוק כפי שרשומה):

• ISTQB Glossary (2023, ALTOM Consulting) — © 2026 ALTOM CONSULTING
ALL RIGHTS RESERVED [36+L3800-L3804] [42+L3769-L3772]

- Ministry of Testing (MoT) – מאמר על All rights reserved 【45†L6-L14】 【45†L120-L127】
Test Charters, 2024. © 2026 Ministry of Testing Ltd.
 - QAblog (ישראל) – מאמרי STD ו- 【6†L28-L31】 【5†L114-L121】 © 2026 QAblog.co.il
Test Case, 2026.
 - Hebrew Wikipedia – ערך "מקרה בדיקה" (2023). הטקסט משוחרר לפי **Creative Commons Attribution-ShareAlike 4.0** 【L7-L10†48】 【50†L1-L4】.
 - Virtuoso QA (בלוג) – מאמר "Test Case – Definition...", 2026. © 2026 SpotQA, Creators of Virtuoso QA
【62†L1217-L1219】 【56†L7-L10】.QA
 - Kualitee – "Forward and Backward Traceability in Testing" (2025). © 2026 Copyright Kualitee. All rights reserved 【18†L351-L354】 【64†L1-L3】
【L85-L93†59】.unclear:Medium (Vladislav S., 2026) – "Test Case Management in 2026...". License
 - John Bryce Jobs (אתר) – שאלות ראיון QA (2018).unclear:QA 【L1-L4†60】.
-